Université Abdelmalek Essaâdi Écoles National des Sciences
Appliquées – Al Hoceima

COMPTE RENDU

TP4-MVC2 : Application Web JEE

Gestion des Produits (MVC2 + Hibernate)

Pr. Mohamed CHERRADI
TP 4-MVC2 JEE + Hibernate + MySQL
TDIA2-S4 / 2025-26

Hiba EL ouazi

1 Introduction

Objectifetcontextedu TP

Ce compte rendu présente la réalisation du **TP N°4-MVC2 : Application Web JEE : Gestion des Produits**, dans le cadre du module **J2EE (Jakarta EE)**. L'objectif est de faire évoluer l'architecture **MVC1** vers **MVC2** (Front Controller Pattern) et de remplacer la persistance en mémoire par une **base de données MySQL** gérée via **Hibernate ORM**. L'application conserve le mécanisme d'**authentification**, la **gestion des rôles** (ADMIN/USER) et les **Filtres Servlet**.

Objectifs du TP

→ Maîtriser l'architecture **MVC2** (Front Controller Pattern) avec Servlet et JSP.

→ Utiliser **Hibernate ORM** pour la persistance des données en base **MySQL**.

→ Implémenter le patron **DAO** avec Hibernate (Session, Transaction, HQL).

→ Conserver le **CRUD** complet avec un seul **DispatcherServlet**.

→ Maintenir le mécanisme d'**authentification** basé sur les sessions HTTP.

→ Maintenir la gestion des **rôles** utilisateurs (ADMIN / USER).

→ Utiliser les **Filtres Servlet** pour la sécurité et le contrôle d'accès.

→ Gérer les erreurs HTTP (**404, 500**) avec des pages personnalisées.

2 Architecture du Projet

Structure MVC2 et flux de données

Le projet repose sur une architecture **3-tiers** respectant le patron **MVC2 (Front Controller)**. Contrairement à MVC1 où chaque action avait sa propre Servlet, MVC2 centralise toutes les requêtes dans un **DispatcherServlet unique** qui redirige vers la méthode handler appropriée selon le paramètre **?action=XXX**. La persistance est assurée par **Hibernate ORM** avec une base **MySQL**.

Couche	Package	Classes principales	Rôle
Model	entities	Produit.java, User.java (@Entity JPA)	Entités JPA mappées sur tables MySQL
DAO	dao	ProduitDAO, ProduitDAOImpl, UserDAO, UserDAOImpl	Accès MySQL via Sessions Hibernate
Service	services	ProduitService, ProduitServiceImpl, UserService	Logique métier, validation
Controller	web	DispatcherServlet, LoginServlet	Front Controller unique (MVC2)

`Filtres`	filters	AuthFilter, RoleFilter	Sécurité et contrôle d'accès
`Vue`	views	login.jsp, index.jsp, error.jsp	Affichage HTML dynamique avec JSTL
`Util`	util	HibernateUtil.java	Singleton SessionFactory Hibernate

Flux de navigation (MVC2) :

Navigateur	→	Dispatcher	→	JSP (View)
Utilisateur		?action=list		index.jsp
Utilisateur		LoginServlet		login.jsp
Admin		?action=add		index.jsp

Chaîne des Filtres Servlet :

→ **AuthFilter (1er)** : intercepte toutes les URLs (/*). Si l'utilisateur n'est pas connecté → redirection vers /login.

→ **RoleFilter (2e)** : intercepte /produits — vérifie le paramètre ?action. Pour add/edit/update/delete → vérifie le rôle ADMIN. Sinon → error.jsp (403).

3 Structure du Projet

Le projet **tp4-MVC2** a été créé comme **Maven Project** dans Eclipse IDE. MVC2 utilise Maven pour gérer les dépendances Hibernate et MySQL. Les JSPs sont dans **src/main/webapp/views/** et les classes Java dans des packages séparés par couche.

Fichier / Package	Emplacement	Rôle
`Produit.java, User.java`	entities/	Entités JPA (@Entity, @Table, @Id...)
`ProduitDAO, ProduitDAOImpl`	dao/	Interface DAO + implémentation Hibernate
`UserDAO, UserDAOImpl`	dao/	Auth via HQL (findByUsernameAndPassword)
`ProduitService, ServiceImpl`	services/	Logique métier + validation
`UserService, UserServiceImpl`	services/	Authentification + init utilisateurs en BDD
`HibernateUtil.java`	util/	Singleton SessionFactory (une seule instance)
`DispatcherServlet.java`	web/	Front Controller MVC2 (routing par ?action=)
`LoginServlet.java`	web/	Authentification GET (form) / POST (verify)
`AppInitializer.java`	web/	Listener : init Hibernate au démarrage

`AuthFilter, RoleFilter`	filters/	Filtres sécurité : auth + contrôle rôles
`hibernate.cfg.xml`	resources/	Config Hibernate : URL MySQL, dialect, DDL
`login.jsp`	views/	Formulaire de connexion
`index.jsp`	views/	Vue principale CRUD adaptative (ADMIN/USER)
`error.jsp`	views/	Page d'erreur HTTP 403/404/500
`web.xml`	WEB-INF/	2 Servlets + 2 Filtres (au lieu de 6+2 en MVC1)
`pom.xml`	racine/	Dépendances Maven : Hibernate, MySQL, JSTL

Structure arborescente dans Eclipse :

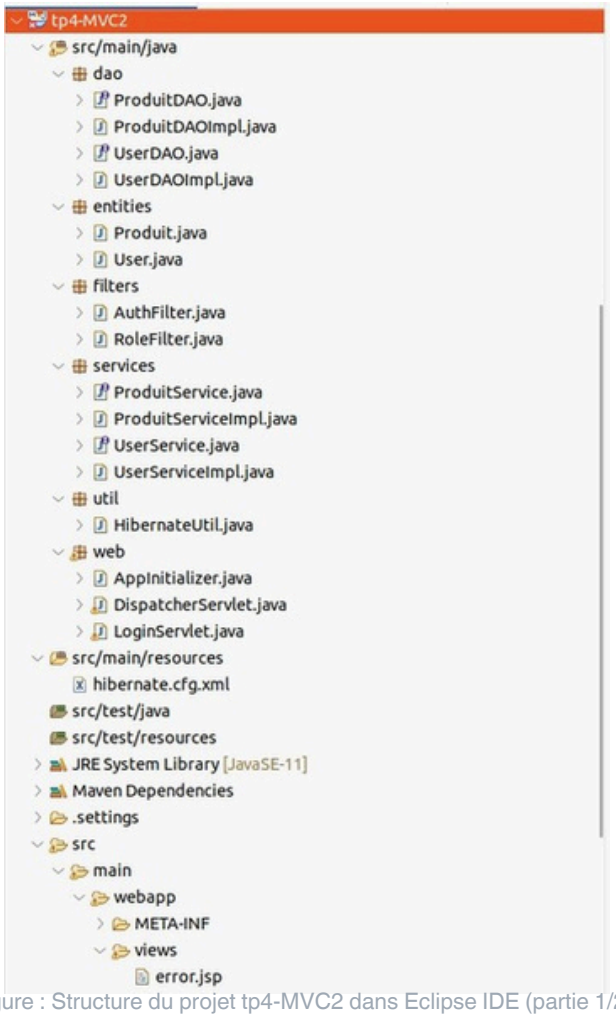

Figure : Structure du projet tp4-MVC2 dans Eclipse IDE (partie 1/2)

Figure : Structure du projet tp4-MVC2 dans Eclipse IDE (partie 2/2)

4 Authentificationet Gestion des Rôles

Sécurité et contrôle d'accès

L'application intègre un système d'**authentification** complet basé sur les **sessions HTTP** Jakarta EE. Contrairement au TP3 (liste en mémoire), les utilisateurs sont maintenant stockés en **base de données MySQL** et la vérification se fait via une requête **HQL Hibernate**.

Mécanisme d'authentification :

→ L'utilisateur soumet son identifiant et mot de passe via **login.jsp**.

→ La **LoginServlet** appelle UserService → UserDAOImpl → requête HQL en base MySQL.

→ En cas de succès : création d'une **session HTTP** (HttpSession) avec l'utilisateur et son rôle.

→ En cas d'échec : message d'erreur affiché dans le formulaire, sans création de session.

→ La session expire automatiquement après **30 minutes** d'inactivité.

→ Au démarrage, **AppInitializer** insère les utilisateurs par défaut en BDD si absents.

Matrice des droits par rôle :

Action	ADMIN	USER
Consulter la liste des produits		
Rechercher par ID ou mot-clé		
Ajouter un produit		✘
Modifier un produit		✘
Supprimer un produit		✘
Déconnexion		

Comptes utilisateurs de test :

Identifiant	Mot de passe	Rôle
`admin`	admin123	**ADMIN**

user	user123	**USER**

5 Captures d'Écran de l'Application

Résultats obtenus

L'application est déployée sur **Apache Tomcat 10.1** et accessible à **http://localhost:8081/tp4-MVC2/**. Les captures suivantes illustrent les différentes fonctionnalités de l'application. Les données sont maintenant **persistées en base MySQL** via Hibernate.

Page de Connexion - login.jsp

Figure : formulaire d'authentification

Erreur d'Authentification

Lorsqu'un utilisateur saisit des identifiants incorrects, un message d'erreur s'affiche dans le formulaire. La vérification se fait en base MySQL via HQL.

Figure : Message d'erreur pour identifiants incorrects

Tableau de Bord : Profil ADMIN

Après connexion ADMIN, l'interface complète est accessible : formulaire d'ajout à gauche, liste des produits avec actions Modifier/Supprimer à droite. Les données proviennent de la base MySQL via Hibernate.

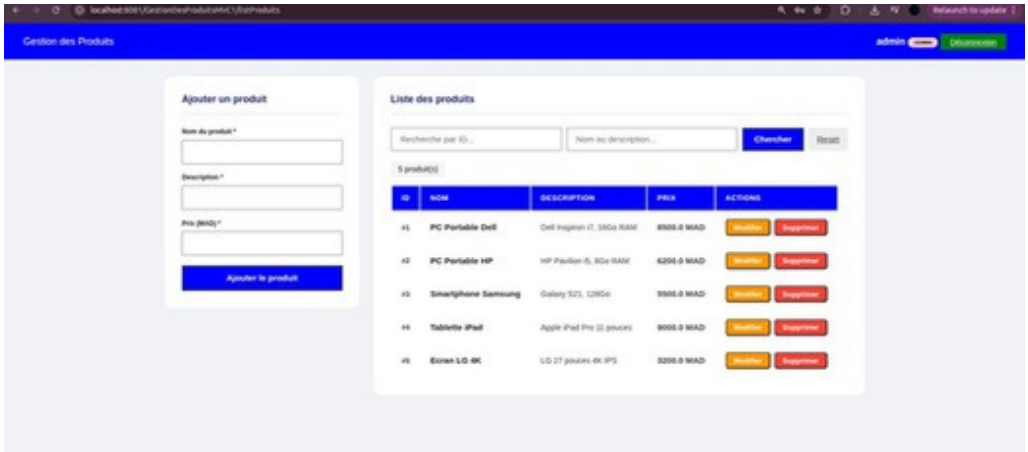

Figure : vue ADMIN avec liste de 5 produits

Ajout d'un Produit

Figure : Formulaire d'ajout rempli avec les données du nouveau produit

Après soumission (pattern Post-Redirect-Get), confirmation en vert et produit ajouté en liste. Le produit est **persisté en base MySQL** via session.persist().

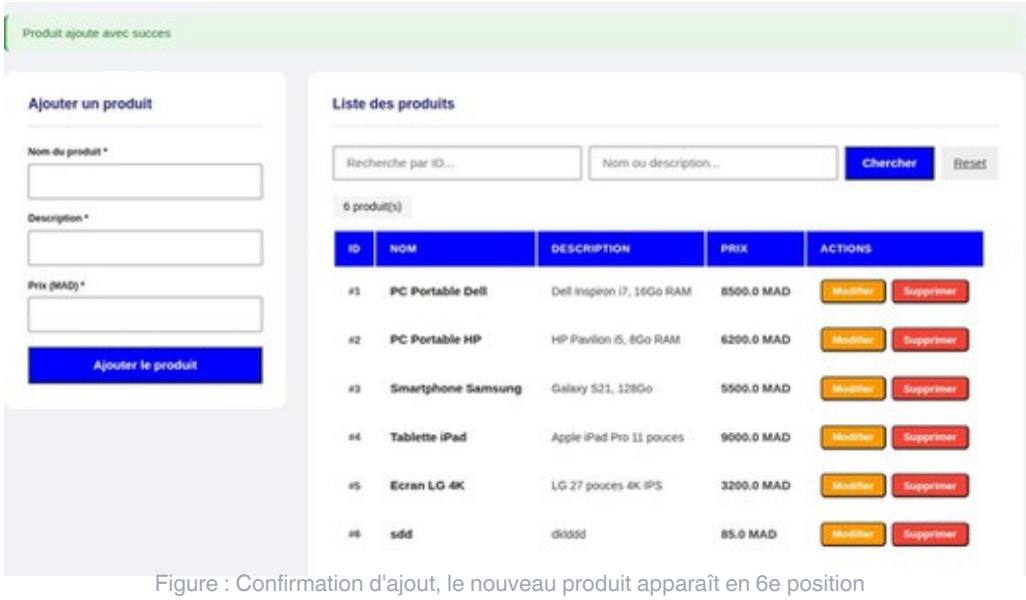

Figure : Confirmation d'ajout, le nouveau produit apparaît en 6e position

Recherche de Produits

Deux modes : recherche par **ID exact** et recherche par **mot-clé** (nom ou description) via requête **HQL LIKE** en base MySQL.

Figure : Recherche par ID exact

Figure : Recherche par mot-clé « pc »

Modification d'un Produit

Figure : Formulaire pré-rempli pour modification

Figure : Confirmation de mise à jour réussie

Suppression d'un Produit

Figure : Boîte de dialogue de confirmation avant suppression

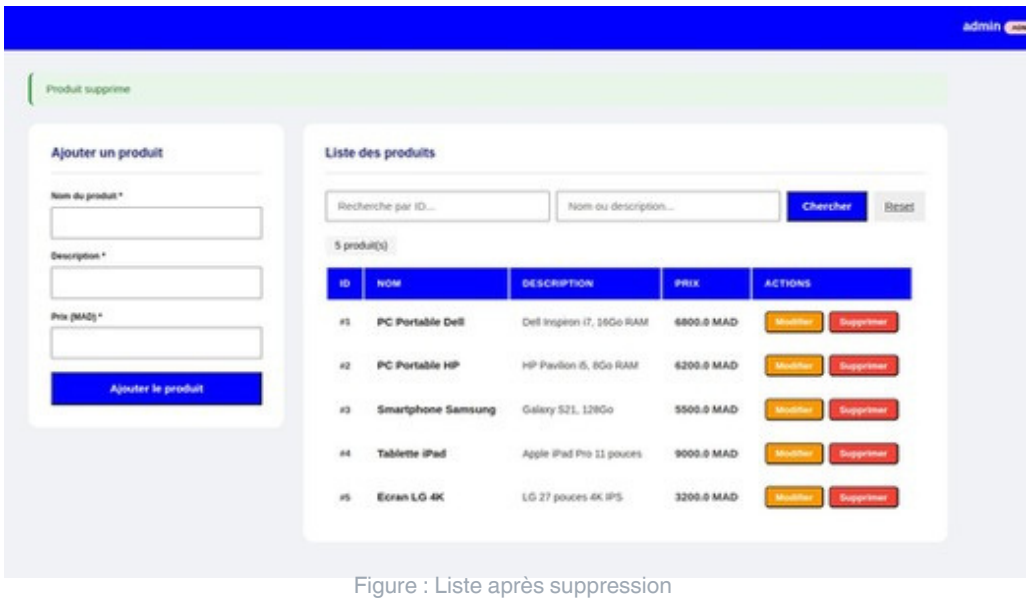

Figure : Liste après suppression

Vue Utilisateur : Profil USER

Avec le rôle USER : le formulaire CRUD est remplacé par un panneau consultation, la colonne Actions est masquée.

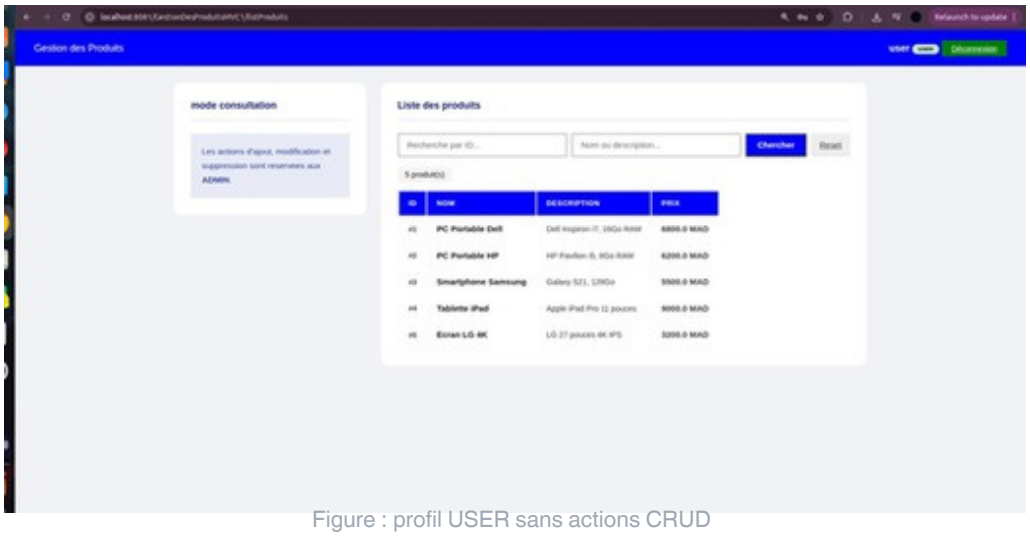

Figure : profil USER sans actions CRUD

Figure : Recherche par ID effectuée par un utilisateur USER

Gestion des Erreurs HTTP

Figure : Page d'erreur 404, URL invalide saisie manuellement

6 Conclusion

Ce TP a permis de faire évoluer l'application de gestion des produits vers une architecture **MVC2** avec persistance **Hibernate + MySQL**. Le passage de MVC1 à MVC2 a réduit le nombre de Servlets de 6 à **2 seulement** (DispatcherServlet + LoginServlet), centralisant le routing dans un Front Controller unique. L'intégration d'Hibernate garantit la **persistance des données** entre les redémarrages du serveur, contrairement à la liste en mémoire du MVC1.